用 OCP 做一个奶茶铺：从零复现指引

🎯 **这份文档写给谁：**完全没听过 OCP 的同学。读完之后，你应该能明白 OCP 是什么、为什么值得自己做一遍，并且照着本文的命令，在自己电脑上跑出一个真实的 OCP 服务。

怎么读：第 1 节是「五个环节」，每个环节只保留一段提问 + 一张反馈截图，展示我们是怎么一步步把一个开放式想法落成可运行代码的。第 2 节是「复现走查」，照着敲就能得到同样的结果。

1. 先花两分钟认识 OCP

OCP 全称 **Open Commerce Protocol（开放商业协议）**。它解决的问题是：让 AI Agent 能自己去发现、查询、解析、触发别人家的商业对象——商品、岗位、房源、政策，而不需要为每一家单独写一套对接代码。

你可以把它想成「商业世界的 USB 接口」：只要对方按同一套约定提供接口，任何 Agent 都能插上去用。

OCP 把职责拆成三层，各管一段，互不越界：

层	负责什么	不管什么
Registration Node	只负责「发现有哪些 Catalog 存在」	不存具体数据
Catalog Node	检索商业对象 + 把它解析成可执行动作	不真正成交
Provider	提供原始数据	不管协议怎么对外

本指引只做中间的 **Catalog Node**——它也是最能体现协议价值的一层。一个合规的 Catalog Node 只需要对外暴露六个端点：

代码块

```
GET  /.well-known/ocp-catalog   发现入口：告诉别人「我在这」
GET  /ocp/manifest              能力声明：我支持哪些检索、有哪些契约
GET  /ocp/health                存活探测
GET  /ocp/contracts             数据契约（ObjectContract）
POST /ocp/query                 检索：返回数据投影
POST /ocp/resolve               解析：把一条结果变成「可执行的动作入口」
```

一句话记住它俩的区别：query 给「投影」（能看不能动），resolve 才给「动作」（可点、可下单）。这是后面演示的高潮。

2. 五个环节：我们是怎么做出来的

下面五轮对话，是从「读协议」到「做出可运行演示」的真实过程。每轮只保留一句原始提问和对应反馈，不重复。

2.1 环节 1 · 先读懂协议

一切从读规范原文开始。不了解协议就动手，做出来的东西大概率不合规。

> **我的提问：**
>
> 去理解一下这个协议：https://github.com/Open-Commerce-Protocol/OCP-Catalog

我的提示词

去理解一下这个协议：https://github.com/Open-Commerce-Protocol/OCP-Catalog

Claude 的反馈

```
OCP = Open Commerce Protocol —— 让 AI Agent 能发现 / 查询 / 解析 / 触发商业对象的开放协议。

三层职责分离：
  · Registration Node   只负责「发现有哪些 Catalog」
  · Catalog Node        负责「检索商业对象 + 解析成动作」
  · Provider            负责「供数」

Catalog Node 的六个端点：
  GET  /.well-known/ocp-catalog    发现入口
  GET  /ocp/manifest               能力声明
  GET  /ocp/health                 存活探测
  GET  /ocp/contracts              数据契约
  POST /ocp/query                  检索（给投影）
  POST /ocp/resolve                解析（给动作）

一句话记住：检索给投影，解析才给动作。
```

环节 1 反馈：OCP 是什么 + 三层职责 + Catalog Node 六个端点

2.2 环节 2 · 立项，先要一份计划

理解了协议之后，提出做一个真实的二次应用来验证它。要求先给计划、批准后再动手——避免方向做偏浪费工。

> **我的提问：**

能不能基于这个协议做一个买奶茶的二次应用开发并做演示？先给我计划，允许后再做开发。

我的提示词

能不能基于这个协议做一个买奶茶的二次应用开发并做演示？先给我计划，允许后再做开发。

Claude 的反馈

```
计划：8 个文件、约 600 行 TypeScript，做一个规范合规的奶茶铺 Catalog Node。

范围（拍板）：
  · 纯 CLI 走查，不做网页
  · 只做 Catalog Node，不做 Registration / Provider
  · 交易落到一个「非协议」mock 端点，显式标注边界

三个设计要点（也是三个论点）：
  1. manifest 只声明已实现的能力（规范 §12）
  2. ObjectContract 用足 field_usage_policy，never_expose 内部成本价（§7）
  3. Resolve 的 purpose 分叉：view→url / checkout→api（§10）

验收标准：官方 CLI 能驱动 + 官方 schema 能校验。
```

环节 2 反馈：计划 8 个文件约 600 行 TypeScript，含范围拍板与三个设计要点

2.3 环节 3 · 代码写完了，怎么跑

开发完成后，关心的是「别人要怎么把它跑起来」。这一步决定了演示能否被复现。

我的提问：

如何演示

我的提示词

如何演示

Claude 的反馈

```
三种演示方式：
  ① 一键走查    bash scripts/demo.sh （起服务 → 10 步官方 CLI → 停服务）
  ② 手动逐条    自己起 bun run start，再一条条敲 ocp 命令
  ③ 自证合规    bun run selfcheck （用官方 schema 校验全部输出）

推荐 ①，退出码 0 即全部通过。
```

环节 3 反馈：三种演示方式，推荐一键走查 demo.sh

2.4 环节 4 · 照做了却跑不动

复现时最常见的两类失败。把排错过程也留在这里，因为新人几乎一定会踩到。

我的提问：

为什么跑不了

我的提示词

为什么跑不了

Claude 的反馈

两个原因：
① 终端不在项目目录 —— 第二个终端停在 ~，脚本不在那儿
解决：先 cd ~/Documents/OCP/work/ocp-milktea-demo
② 端口冲突 —— 第一个终端已经占用了 4400
解决：先停掉，或让 demo.sh 复用已在跑的服务

环节 4 反馈：两个原因——终端不在项目目录、端口 4400 被占用

2.5 环节 5 · 能不能接别家已有的 MCP

最后是一个真实世界的延伸问题：如果商家已经提供了 MCP 接口，OCP 还有用吗？

我的提问：

瑞幸有自己的 MCP 接口支持点单了，能不能用 OCP 进行接入来实现我点单

我的提示词

瑞幸有自己的 MCP 接口支持点单了，能不能用 OCP 进行接入来实现我点单

Claude 的反馈

可以接入，但是「叠上去」，不是「换掉」。

```
  瑞幸 MCP  tools:  searchProductForMcp / previewOrder / createOrder …
  OCP 端点    检索菜单 + 解析动作，把 Agent 送到门口
```

分层：

```
  OCP = 协议层（发现 / 检索 / 契约 / 权限）
  MCP = 适配器层（规范 protocol-boundaries.md 明确列出）
```

正确做法：写一个 OCP Catalog 包住瑞幸 MCP，

```
  checkout 返回 action_type='workflow'（而不是 'api'），
  真正的下单由 Agent 用用户自己的 token 在 OCP 之外完成。
```

三个硬冲突：

```
  鉴权    每用户 Bearer token  vs  公开的 ActionBinding
  状态    JSON-RPC 会话         vs  一个 {url, method}
  语义    私有工具名           vs  统一 QueryPack / ObjectContract
```

环节 5 反馈：可以接入，但是「叠上去」不是「换掉」——OCP 是协议层，MCP 是适配器层

💡 **这一节最该记住的一句话：**MCP 在 OCP 规范里属于 **适配器层**（adapter layer），和 CLI、Skill、插件并列，**不是协议层**。所以接别家 MCP 的正确姿势是写一个 OCP Catalog 把它包起来，而不是用 MCP 去替换 OCP。

3. 复现走查：十步，从起服务到下单

下面十步全部由**官方未修改的工具链**驱动。前置条件只有两个：装好 `bun`（1.3+），以及全局安装官方 CLI。

代码块

```
# 前置：安装官方 OCP CLI（v0.1.3）
npm i -g @ocp-catalog/ocp-cli

# 进入项目目录
cd work/ocp-milktea-demo
bun install
```

然后一键走查，它会把服务起起来、跑完十步、再停掉：

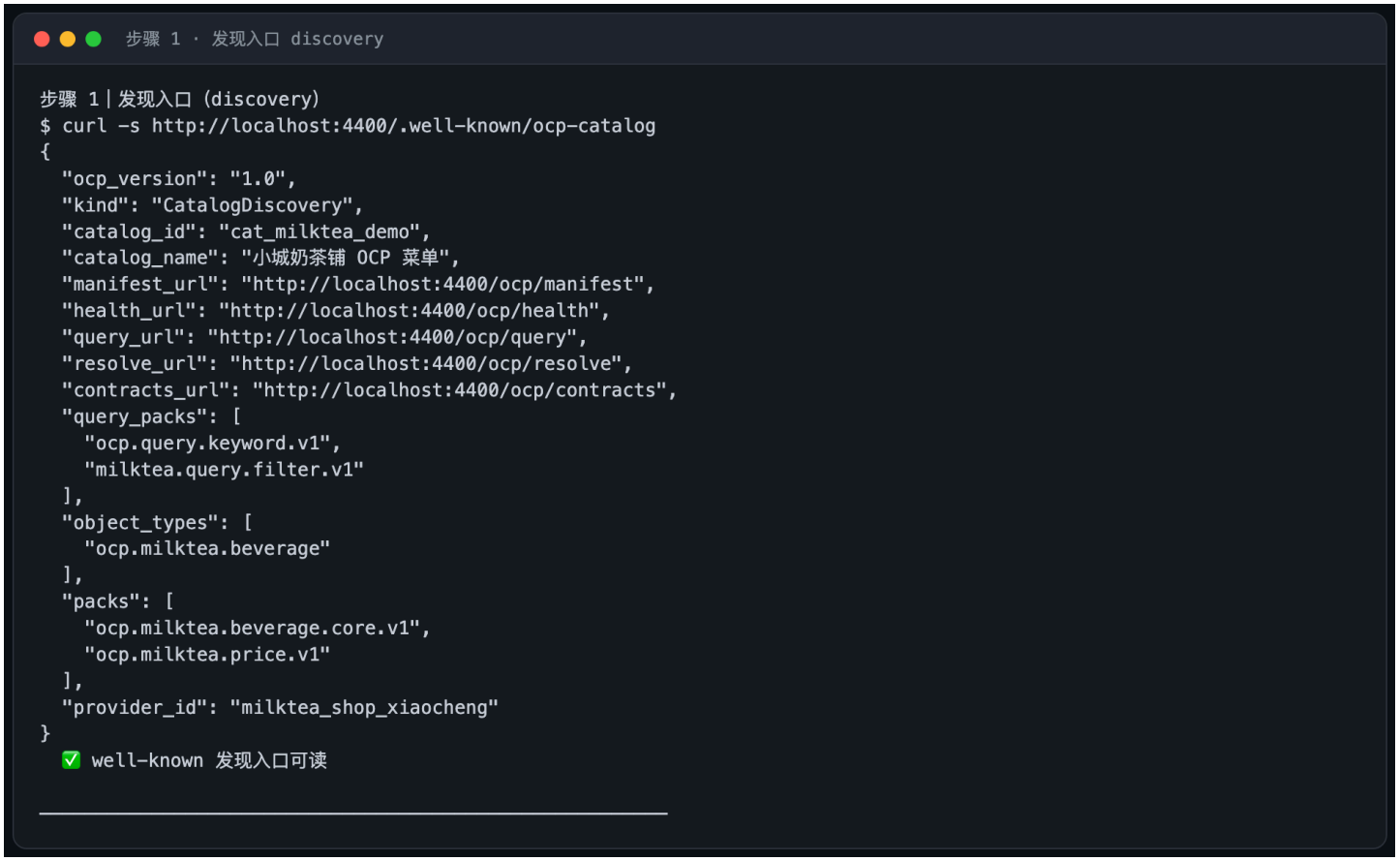
代码块

```
bun run demo
```

💡 如果你的终端 **不在项目目录**，或者 **4400 端口已经被占用**，就会像环节 4 那样跑不动。先 `cd` 到项目目录，并确认没有另一个服务占着 4400。

3.1 步骤 1 · 发现入口

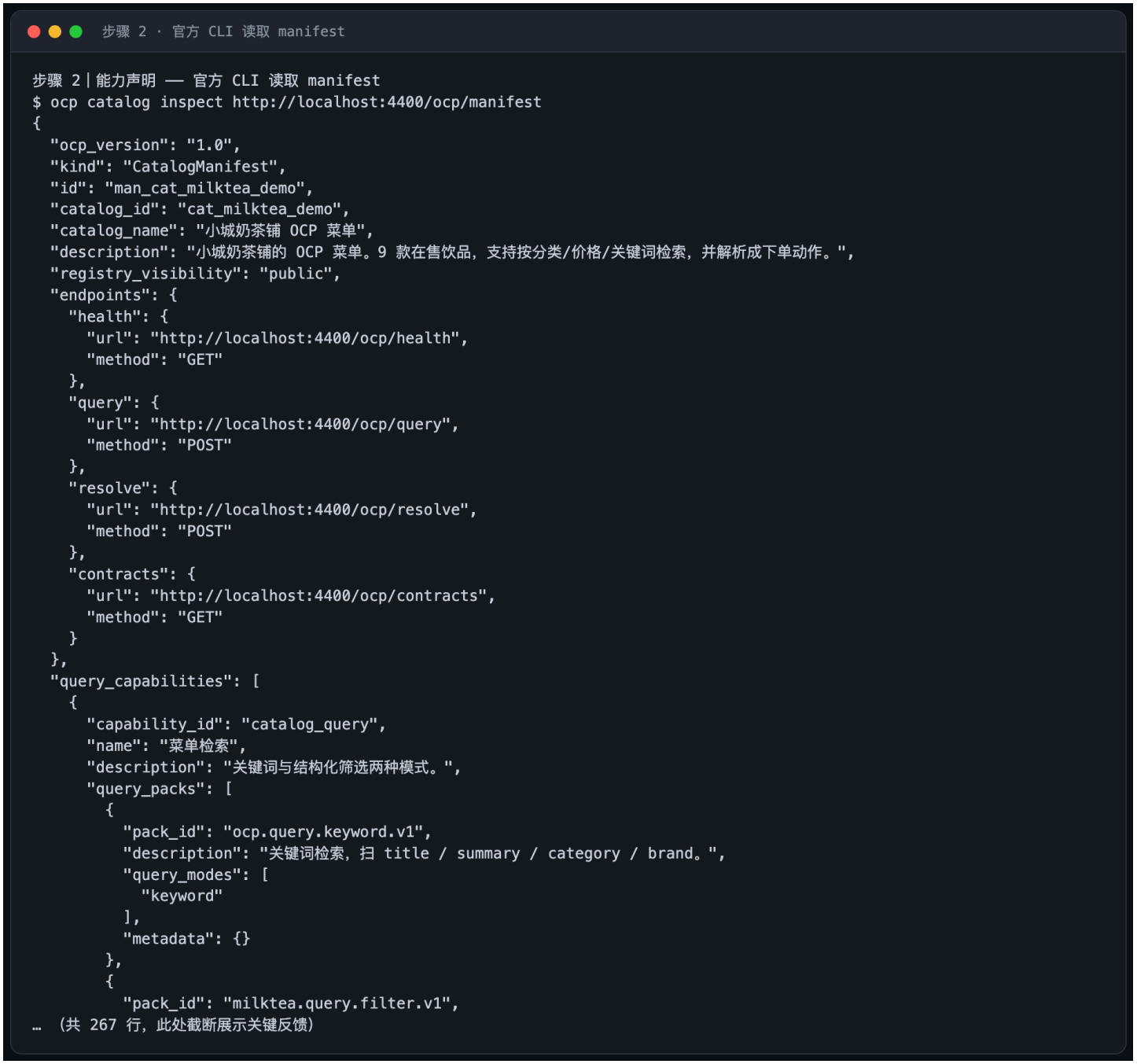
Agent 的第一步是「知道这里有个 Catalog」。访问 well-known 地址即可拿到全部入口。



步骤 1：GET /.well-known/ocp-catalog 返回发现信息

3.2 步骤 2 · 官方 CLI 读取能力声明

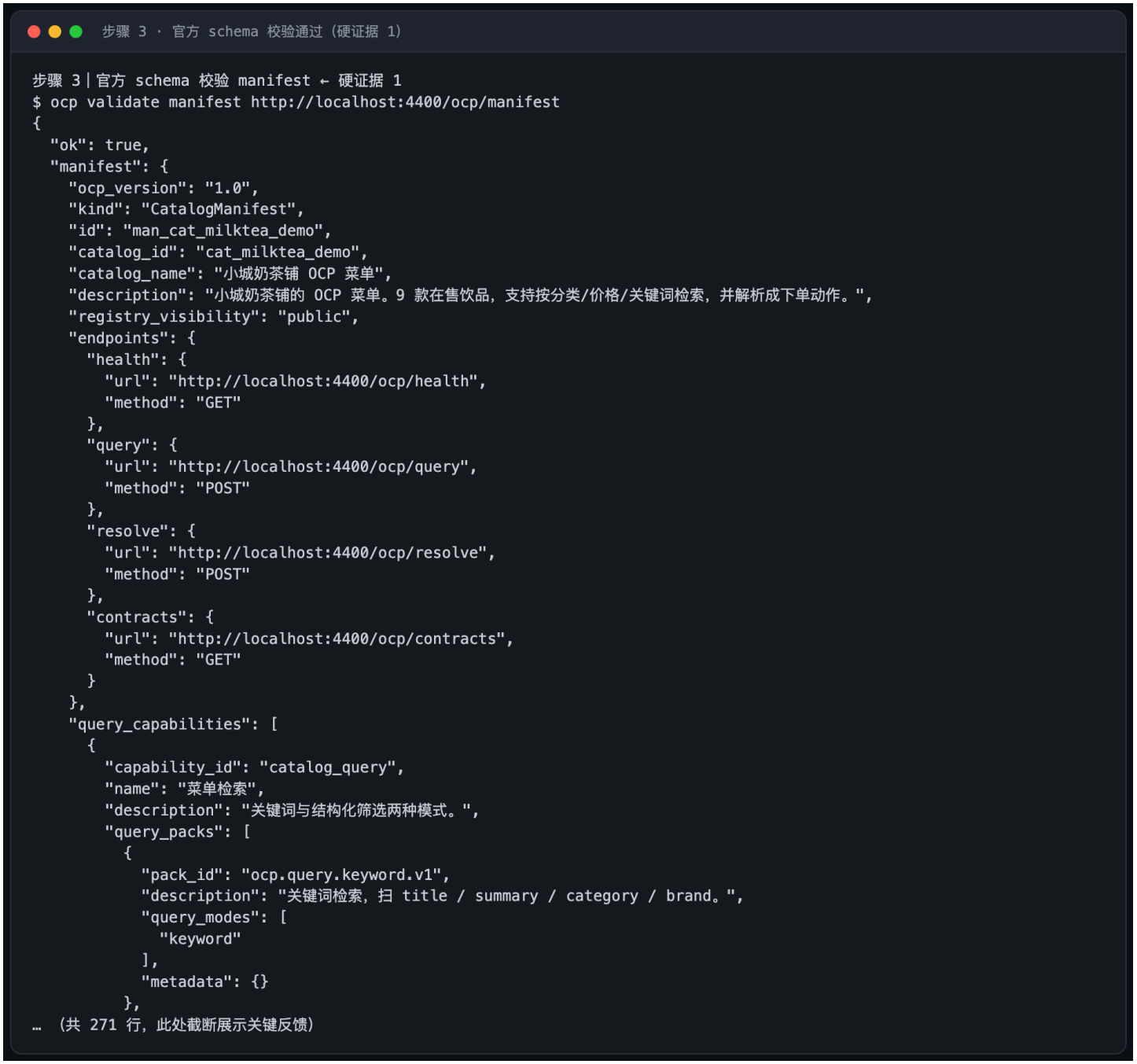
用官方 CLI 读取 manifest，确认它认得我们的服务。这一步证明我们的输出格式是对的。



步骤 2：ocp catalog inspect 读取 manifest

3.3 步骤 3 · 官方 schema 校验通过（硬证据 1）

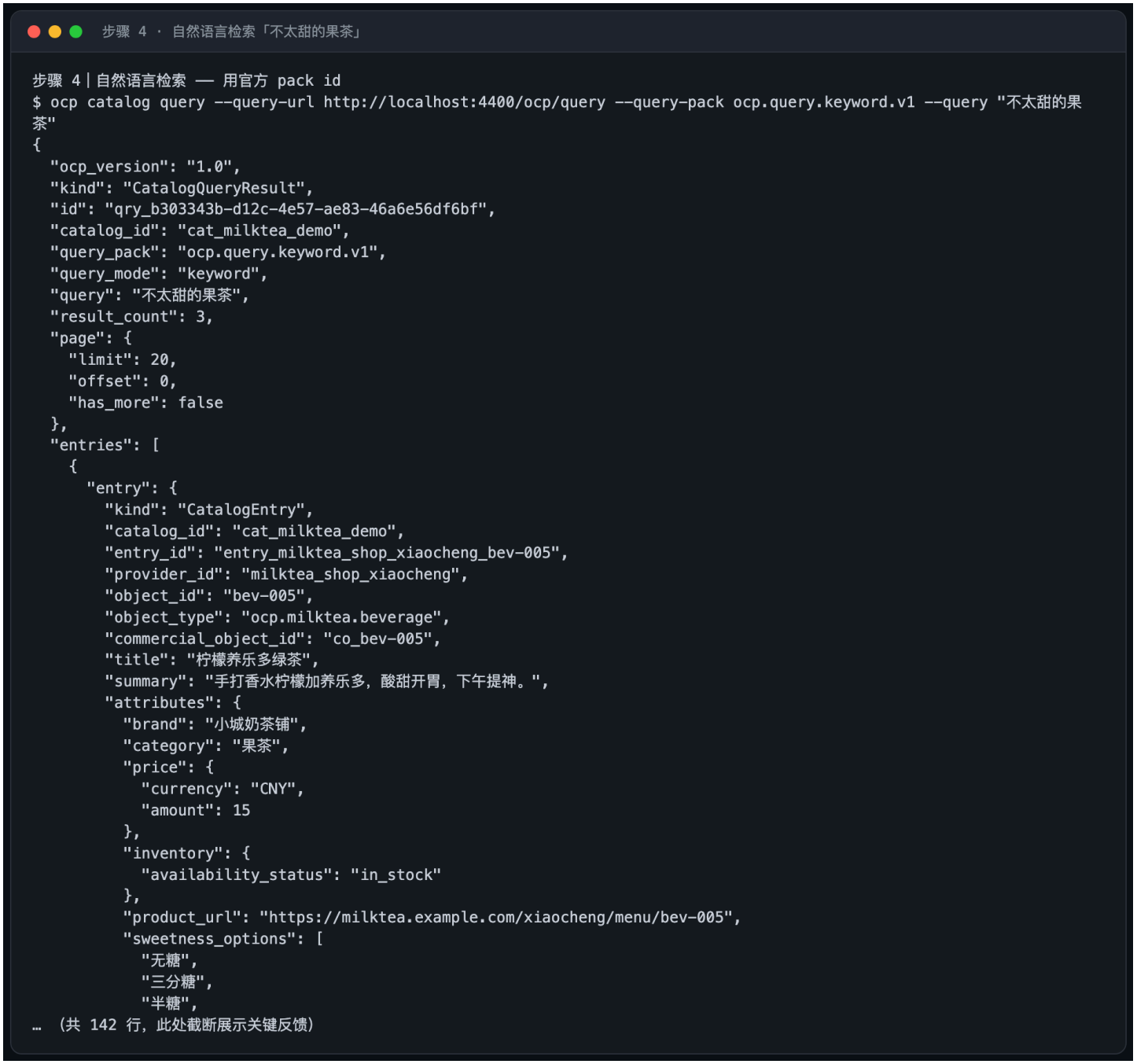
这不是「我说自己合规」，而是官方工具链给出的结论。



步骤 3：ocp validate manifest 返回 ok:true

3.4 步骤 4 · 自然语言检索

用一句人话查菜单。注意这里用的 `ocp.query.keyword.v1` 与官方示例**同一个 pack id**，所以天然互通。



步骤 4：检索「不太甜的果茶」并命中一条

3.5 步骤 5 · 检索前的本地校验

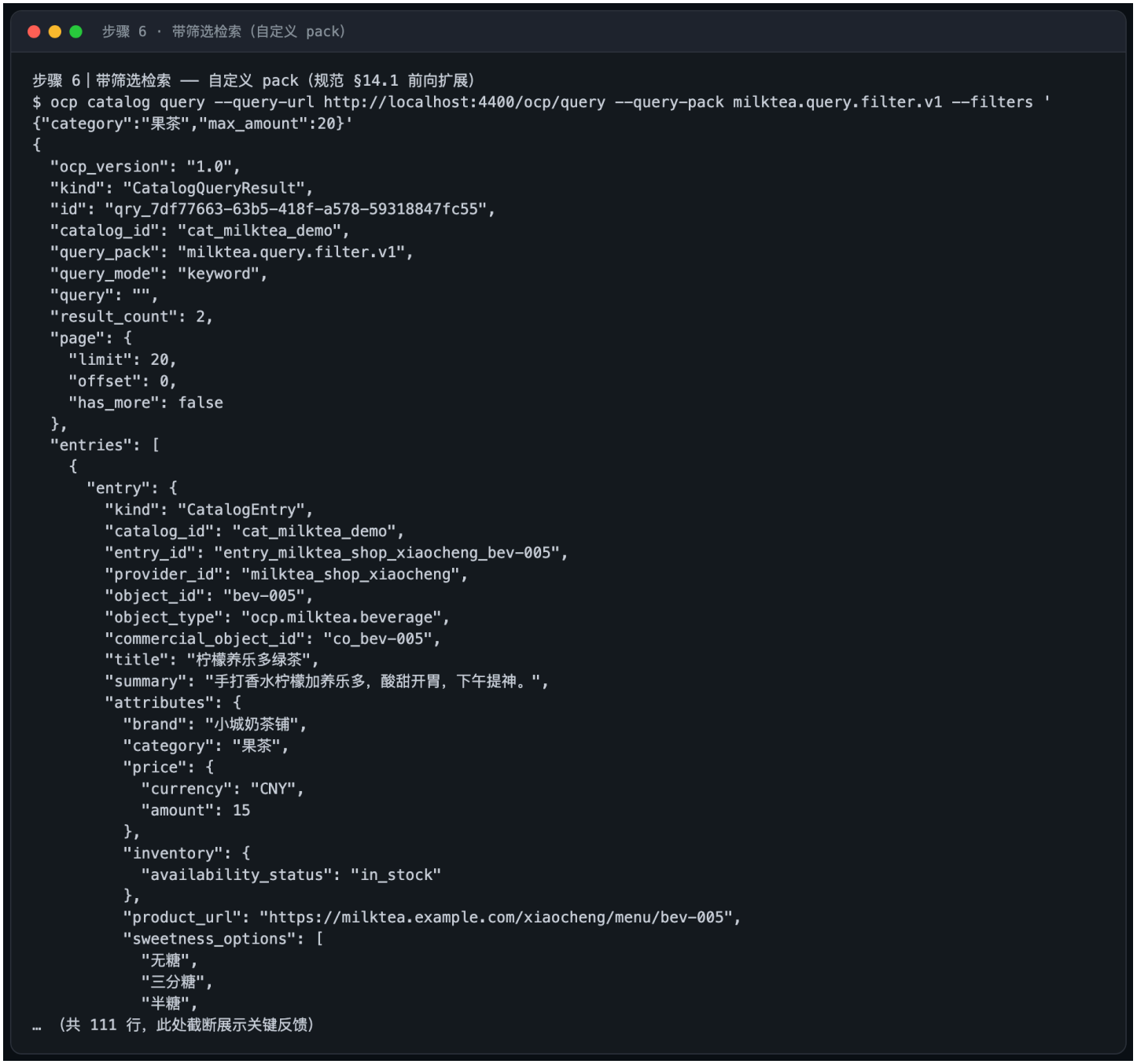
官方 CLI 不联网也能校验请求是否合法——用于提前发现「用了没声明的能力」这类错误。



步骤 5：校验自定义 query pack 的请求体

3.6 步骤 6 · 带结构化筛选的检索

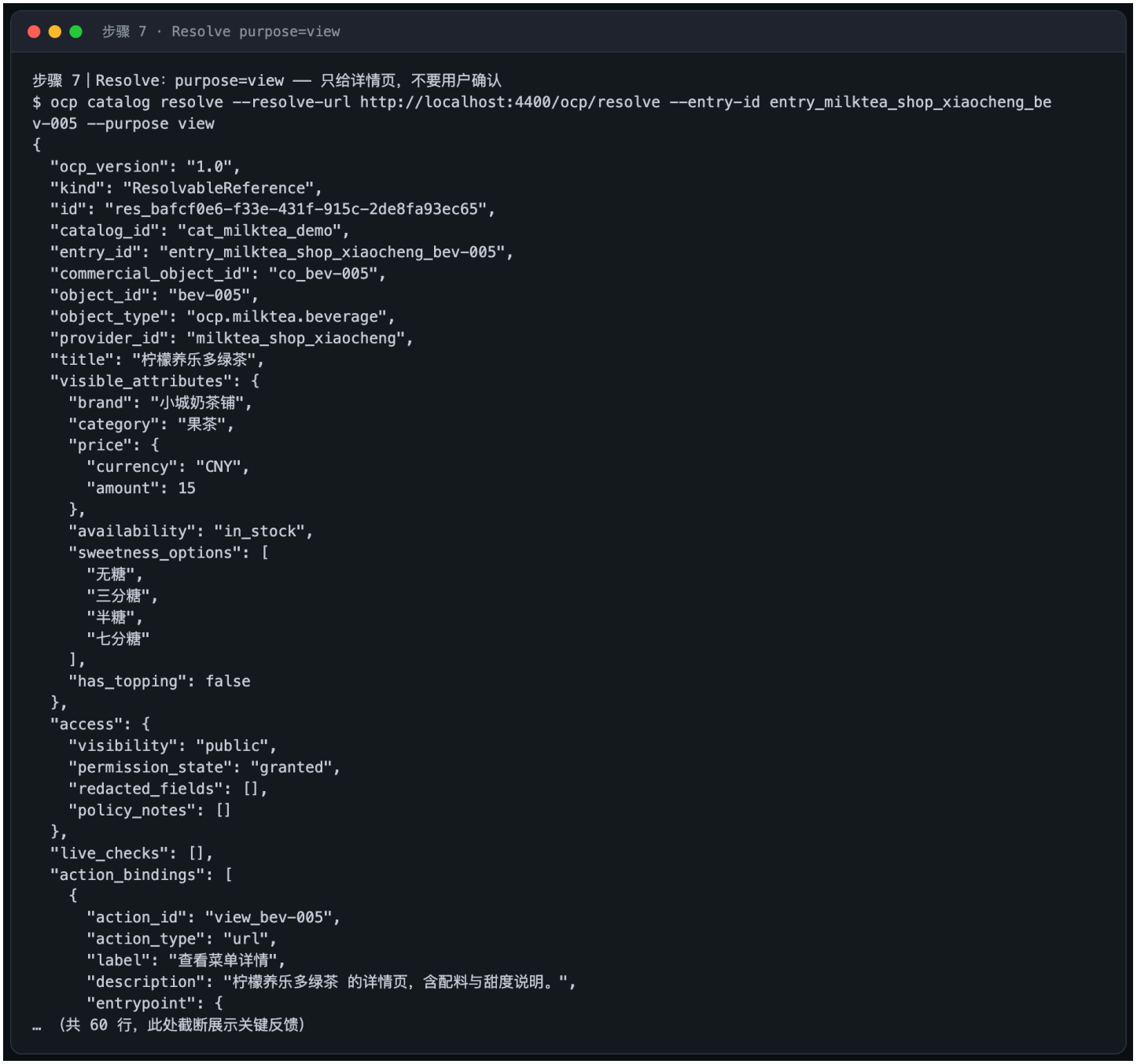
换一个**自定义**的 pack 做筛选（分类 + 价格上限）。这演示了规范里的「前向扩展」：新增能力不必改协议本身。



步骤 6：filter pack 检索「果茶且 20 元以内」

3.7 步骤 7 · Resolve：只想看看详情

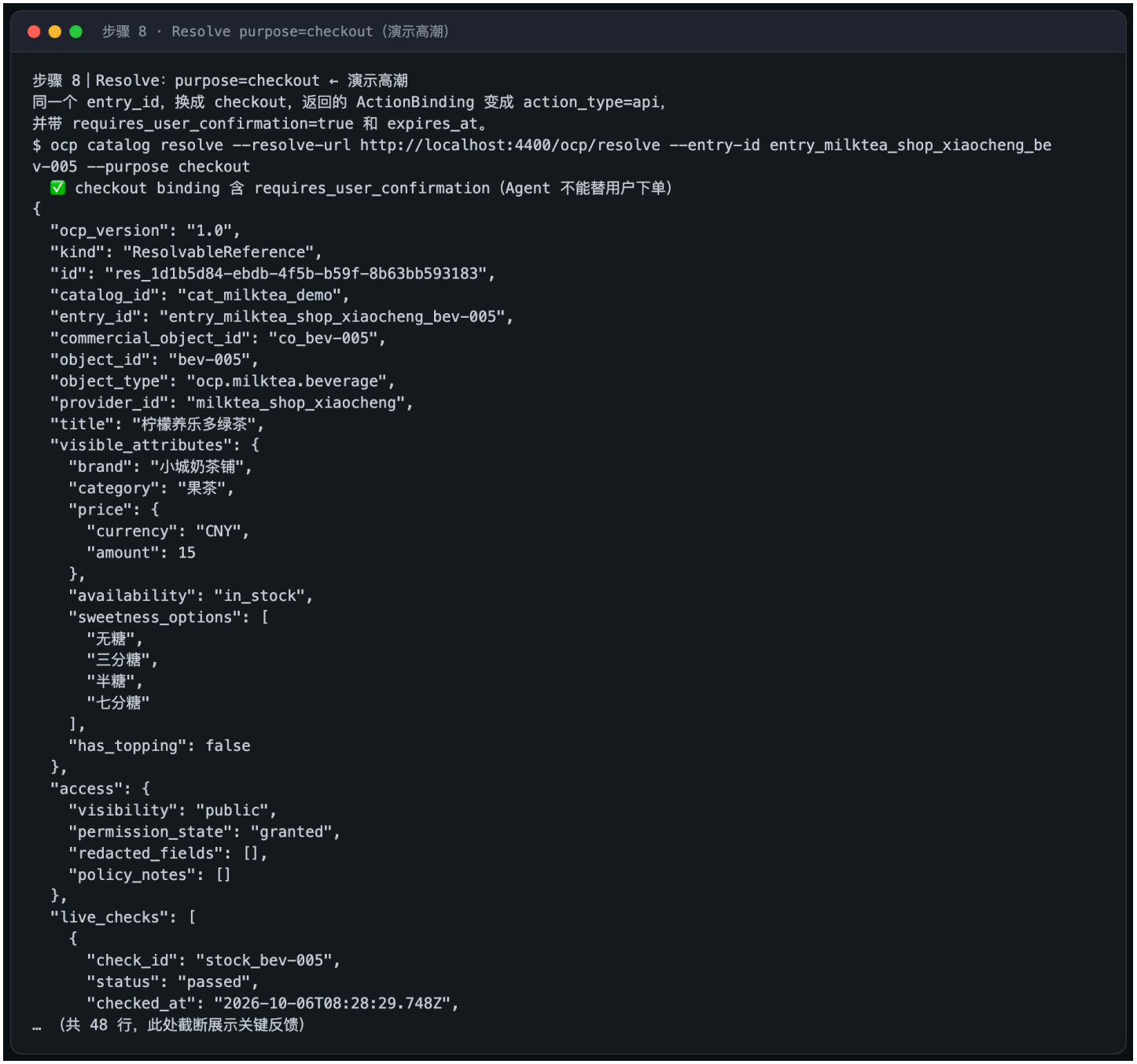
对同一条结果，先以 `purpose=view` 解析。返回的是一个普通详情页链接，不需要用户确认。



步骤 7：purpose=view 返回 url 类型的 ActionBinding

3.8 步骤 8 · Resolve：要下单（演示高潮）

同一个 `entry_id`，只把 purpose 换成 `checkout`，返回的动作就变成了一个 API 下单入口，并且强制带上**用户确认**与**时效**。



步骤 8：purpose=checkout 返回 api 动作 + requires_user_confirmation + expires_at

⭐ **这就是本文最核心的设计点：**同一条数据，看和买返回两个完全不同的动作。查询只给「投影」，解析才给「动作」，而且做动作必须用户确认、还会过期。一个 Catalog 层能同时做到数据过滤和动作约束——这不是转发接口能替代的。

3.9 步骤 9 · 真正下单（边界之外）

最后真正下了一单。注意这个下单端点**不属于 OCP 协议**，它是门店自己的接口——演示「协议负责把 Agent 送到门口，交易本身不归协议管」。



步骤 9：非协议 mock 下单端点返回订单号

3.10 步骤 10 · 边界用例

一个合格的实现还要在出错时守规矩：不存在的 ID 返回 404，规范不支持的翻页返回 400，用了没声明的能力返回 400。



步骤 10：404 / 400 等边界情形均按规范返回

4. 两条硬证据

整个演示的说服力，落在两条可复现的证据上。它们的作用是把「这个方案行不行」从观点争论，变成一条能自己跑出来的事实。

4.1 证据 1 · 官方 schema 认可我们的每一个响应体

用协议官方发布的 schema 包，逐个校验 manifest / health / query / resolve / 契约，连「内部字段有没有泄漏」都单独扫一遍。

代码块

```
bun run selfcheck
```

自证：用官方 schema 校验全部输出

```
用官方 @ocp-catalog/ocp-schema 校验本 Catalog 的响应体

  ✅ CatalogManifest
  ✅ CatalogHealth
  ✅ ObjectContract[0]
  ✅ /ocp/contracts 内契约[0]

never_expose 字段泄漏检查
  ✅ CatalogQueryResult（全量）
  ✅ query 响应不含 cost_amount（9 条全部通过）
  ✅ CatalogQueryResult（不太甜的果茶）
  ✅ CatalogQueryResult（filter pack）

  ✅ ResolvableReference ×18（9 款 × view/checkout）全部通过

checkout 动作约束检查
  ✅ checkout 返回 api 动作 + requires_user_confirmation=true + expires_at

发现入口字段检查
  ✅ /.well-known/ocp-catalog 关键字段齐全

结果：11 通过
```

证据 1：11 项全部通过，均通过官方 @ocp-catalog/ocp-schema 校验

4.2 证据 2 · 一键走查全流程通过

从起服务到停服务，十步全部走完，退出码为 0。

scripts/demo.sh 一键走查结果

```
步骤 1  发现入口 discovery
步骤 2  官方 CLI 读取 manifest        ← 官方工具链认可
步骤 3  官方 schema 校验 manifest     ← 硬证据 1
步骤 4  自然语言检索「不太甜的果茶」
步骤 5  查询前置校验（本地，不发请求）
步骤 6  带筛选检索（自定义 pack）
步骤 7  Resolve  purpose=view
步骤 8  Resolve  purpose=checkout     ← 演示高潮
步骤 9  下单（非协议端点，标注边界）
步骤 10 边界用例：404 / 400 / 400

结果：13 通过   全流程通过 ✅   退出码 0
```

证据 2：13 项断言全部通过，全流程通过

5. 自己动手跑一遍

把上面这些串起来，你需要执行的命令就是这几条。建议按顺序来，出错就回到环节 4 排查。

代码块

```
# 1. 进入项目
cd work/ocp-milktea-demo

```

```
# 2. 装依赖
bun install

# 3. 先自证合规（不启服务）
bun run selfcheck

# 4. 一键走查：起服务 → 十步官方 CLI → 停服务
bun run demo

# 5. 如果想手动一步步来
bun run start        # 另开一个终端，先起服务
# 然后照着第 2 节第 1~10 步的命令逐条执行
```

✅ **跑通的标准：** `bun run selfcheck` 显示 11 项全绿；`bun run demo` 结束时打印「全流程通过」，退出码为 0。

看到这两条，你就完整复现了这个演示——一个第三方按规范写的 Catalog Node，能被未经修改的官方工具链直接驱动。